

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING

—o0o—



LABORATORY REPORT

Waveform generator

SUPERVISOR: Nguyễn Tuấn Hùng

SUBJECT: Digital signal processing on FPGA

GROUP: 08

List of Members

STT	MSSV	Họ Và Tên	Lớp
1		Đoàn Văn Sang	L01
2	2210780	Nguyễn Đại Đồng	L01
3		Nguyễn Thị Trâm Ánh	L01

Ho Chi Minh, ../../20..

Mục lục

Câu 1	1
a)	1
b)	3
c)	4
 Câu 4	 8
a)	8
b)	11
c)	14

Danh sách hình vẽ

1	Thiết kế mạch xấp xỉ chia 3.	2
2	Bộ cộng CSA 8 bit.	8
3	Bộ cộng CRA 4 bit.	8
4	Full Adder.	9
5	Khối tính toán song song 4 bit.	9
6	Bộ cộng CSA 8 bit thiết kế.	10
7	Bộ cộng CSA 32 bit.	10

Danh sách bảng

List of Listings

1	HDL mô tả thiết kế bộ xấp xỉ chia 3.	3
2	HDL mô tả thiết kế bộ xấp xỉ chia 3.	4

3	Kết quả kiểm định cho bộ xấp xỉ chia 3.	5
4	HDL mô tả thiết kế bộ xấp xỉ chia 3.	6
5	Kết quả kiểm định cho bộ xấp xỉ chia 3.	7
6	HDL mô tả thiết kế Full Adder 1 bit.	11
7	HDL mô tả thiết kế CRA 4 bit.	11
8	HDL mô tả thiết kế CSA linh động.	12
9	Kiểm định giành cho CSA.	14
10	Kết quả kiểm định cho thiết kế CSA 8 bit.	15
11	Kết quả kiểm định cho thiết kế CSA 32 bit.	16

Câu 1

Thiết kế mạch tổ hợp xấp xỉ phép chia 3.

Để thực hiện phép chia 3, ta có thể sử dụng phương pháp nhân số đó với $\frac{1}{3}$.

Trong đó, ta có thể xấp xỉ $\frac{1}{3} \approx 2^{-2} + 2^{-4} + 2^{-6} + 2^{-8}$

Cho các stand cell là: cổng NOT, các LOGIC GATE 2 ngõ vào, MUX 2-1, MUX 4-1.

a) Cho ngõ vào là số 32 bit. Thiết kế mạch theo phương pháp đã cho và chỉ sử dụng các standard cell trên.

Đề bài không yêu cầu thiết kế xấp xỉ phép chia 3 cho số có dấu (bù hai) hoặc số không dấu. Vì vậy, nhóm chọn thiết kế cho số có dấu.

Ý tưởng ban đầu:

- Các phép $a \times 2^{-n}$ có thể hiểu là những phép $a \ggg n$ (dịch phải có dấu).

- Vì thế $a \times \frac{1}{3} \approx (a \ggg 2) + (a \ggg 4) + (a \ggg 6) + (a \ggg 8)$.

$\Rightarrow$ Thiết kế cần 3 bộ cộng kèm các phép ghép và lặp số để thực hiện.

Tối ưu ý tưởng:

- Từ hàm số ban đầu: $2^{-2} + 2^{-4} + 2^{-6} + 2^{-8} = 2^{-2} + 2^{-4} + 2^{-4} \times (2^{-2} + 2^{-4})$

- Vì thế $a \times \frac{1}{3} \approx (a \ggg 2) + (a \ggg 4) + (((a \ggg 2) + (a \ggg 4)) \ggg 4)$

- Đặt tham số $N = (a \ggg 2) + (a \ggg 4) \Rightarrow a \times \frac{1}{3} \approx N + (N \ggg 4)$

$\Rightarrow$ Thiết kế cần 2 bộ cộng: 1 để tính toán tham số N và 1 để tính toán $N + (N \ggg 4)$.

$$* N = (a \ggg 4) + (a \ggg 2)$$

$$* S = (N \ggg 4) + N$$

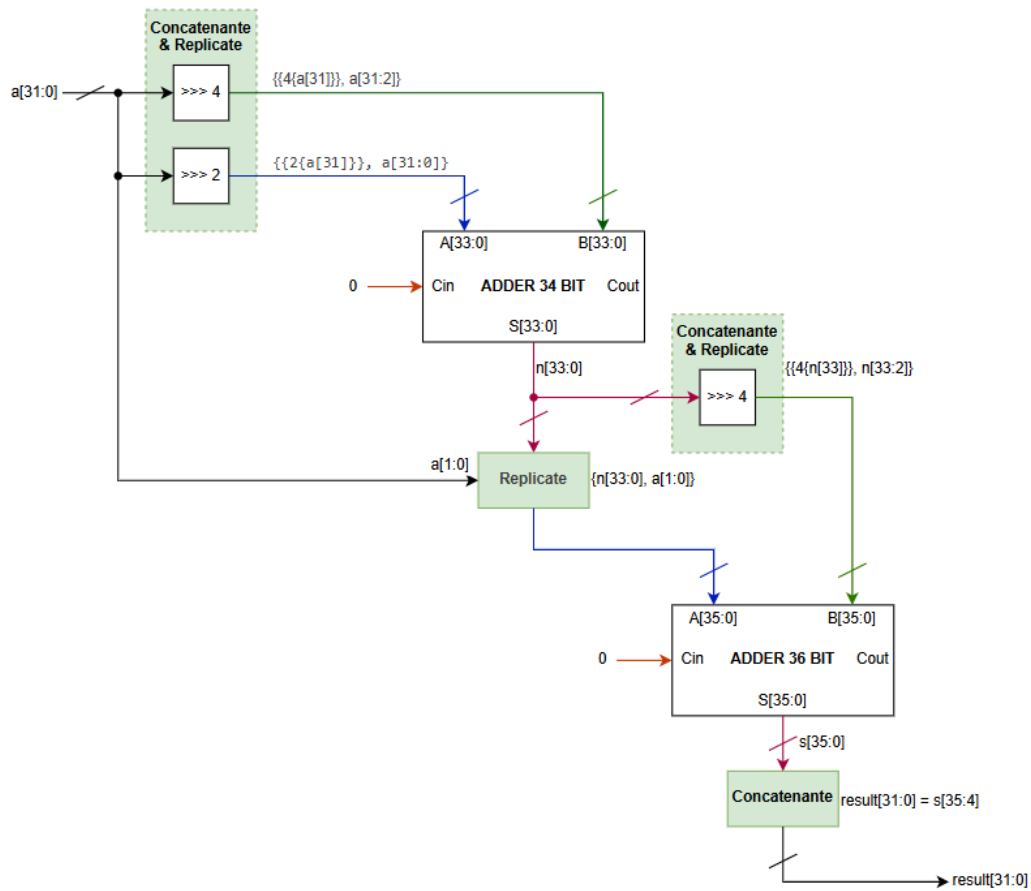
$$\begin{array}{r} a_{31} \ a_{31} \ a_{31} \ a_{31} \ a_{31} \ a_{30} \ \dots \ a_3 \ a_2 \ a_1 \ a_0 \\ + a_{31} \ a_{31} \ a_{31} \ a_{30} \ a_{29} \ a_{28} \ \dots \ a_1 \ a_0 \\ \hline n_{33} \ n_{32} \ n_{31} \ n_{30} \ n_{29} \ n_{28} \ \dots \ n_3 \ n_2 \ n_1 \ n_0 \end{array}$$

$$\begin{array}{r} n_{33} \ n_{33} \ n_{33} \ n_{33} \ n_{33} \ n_{32} \ \dots \ n_5 \ n_4 \ n_3 \ n_2 \ n_1 \ n_0 \\ + n_{33} \ n_{32} \ n_{31} \ n_{30} \ n_{29} \ n_{28} \ \dots \ n_1 \ n_0 \\ \hline s_{37} \ s_{36} \ s_{35} \ s_{34} \ s_{33} \ s_{32} \ \dots \ s_5 \ s_4 \ s_3 \ s_2 \ s_1 \ s_0 \end{array}$$

$\Rightarrow$ Full Adder 34 bit.

$\Rightarrow$ Full Adder 36 bit.

Kết quả ngõ ra Result là 32 bit cao của S.



Hình 1: Thiết kế mạch xấp xỉ chia 3.

Thiết kế các bộ cộng sẽ được trình bày tại Câu 3 và Câu 4.

b) Viết chương trình HDL mô tả mạch đã cho.

```
1  module div_3_estimate (
2      input  logic [31:0] i_a ,
3      output logic [31:0] o_result
4  );
5
6      logic [33:0] n ;
7      logic [35:0] n_sr2, result;
8
9      cra #(
10         .WIDTH  (34)
11     ) stage_i (
12         .i_a      ({2{i_a[31]}}, i_a[31:0]),
13         .i_b      ({4{i_a[31]}}, i_a[31:2]),
14         .i_cin    (1'b0),
15         .o_s      (n),
16         .o_cout   ()
17     );
18
19     assign n_sr2 = {{4{n[33]}}, n[33:2]};
20
21     csa #(
22         .WIDTH  (36), // WIDTH must %4 == 0 (and >= 4)
23         .CLA_CRA(0)  // 1 use CLA 4 bit , 0 use CRA 4 bit
24     ) stage_ii (
25         .i_a      ({n, i_a[1:0]}),
26         .i_b      (n_sr2),
27         .i_cin    (1'b0),
28         .o_s      (result),
29         .o_cout   ()
30     );
31
32     assign o_result = result[35:4];
33
34 endmodule
```

Listing 1: HDL mô tả thiết kế bộ xấp xỉ chia 3.

c) Viết testbench cho mạch, thực hiện testbench với 100 mẫu và tính scoreboard của 100 mẫu đó.

Nhóm tiến hành kiểm định cho thiết kế bằng phương trình xấp xỉ mà đề bài đã cung cấp, vì các hệ số của phương trình là số thực nên nhóm tiến hành tạo ngẫu nhiên 100 giá trị A đầu vào thiết kế là dạng integer.

Tiến hành ép kiểu A sang dạng shortreal để nhân với $(2^{-2} + 2^{-4} + 2^{-6} + 2^{-8})$. Sau đó kết quả được ép kiểu lại về dạng integer được sử dụng để so sánh với giá trị đầu ra của thiết kế.

```
1  module div_3_tb();
2
3      parameter TEST_LOOP = 100;
4      parameter real ESTIMATE = (2.0**-2) + (2.0**-4) + (2.0**-6) + (2.0**-8);
5
6      int value_real;
7
8      logic [31:0] logic_value, logic_result;
9
10     logic [31:0] operator_div;
11
12     int value_begin;
13     shortreal trans_value;
14
15     div_3_estimate dut_2 (
16         .i_a      (logic_value),
17         .o_result(logic_result)
18     );
19
20     initial begin
21         $shm_open("waves.shm");
22         $shm_probe("ASM") ;
23     end
24
25     initial begin
26         for(int i = 0; i < TEST_LOOP; i++) begin
27             value_begin = $urandom_range(0, 2**32 - 1);
28             logic_value  = value_begin;
29             trans_value  = $itor(value_begin);
30             value_real   = int'(trans_value * ESTIMATE);
31
32             #10
33             printf_compare("DIV_3", logic_result, value_real);
34         end
35         report();
36
37         $finish;
38     end
39
40     int test_count = 0;
41     int test_pass  = 0;
42
43     task printf_compare (input string type_tb, input int tb_value, input int dut_value);
44         $display("[TIME: %5t] [%s] - %4s: Expect: %8h, DUT: %8h ", $time, type_tb, (tb_value == dut_value) ?
45         "PASS" : "FAIL", tb_value, dut_value);
46         test_count = test_count + 1;
47         test_pass  = (tb_value == dut_value) ? test_pass + 1 : test_pass;
```

```

47     endtask
48
49     task report();
50         $display("\n=====");
51         $display("=====TEST SUMMARY=====");
52         $display("Total test cases: %6d      ", test_count);
53         $display("Passed           : %6d      ", test_pass);
54         $display("Failed            : %6d      ", test_count - test_pass);
55         $display("Pass rate       : %0.2f%%", (test_pass * 100.0) / (test_count));
56         $display("=====");
57     endtask
58
59 endmodule

```

Listing 2: HDL mô tả thiết kế bộ xấp xỉ chia 3.

Kết quả:

```

xcelium> run
[TIME:    10] [DIV_3] - FAIL: Expect: 0b0a6d1b, DUT: 0b0a6d13
[TIME:    20] [DIV_3] - FAIL: Expect: 24586803, DUT: 245867f1
[TIME:    30] [DIV_3] - FAIL: Expect: e33779e8, DUT: e33779eb
[TIME:    40] [DIV_3] - FAIL: Expect: f4310e8d, DUT: f4310e8c
[TIME:    50] [DIV_3] - FAIL: Expect: 060783d8, DUT: 060783d9
[TIME:    60] [DIV_3] - FAIL: Expect: e655eca5, DUT: e655ecac
[TIME:    70] [DIV_3] - FAIL: Expect: 174f6068, DUT: 174f6064
[TIME:    80] [DIV_3] - FAIL: Expect: df44cb4b, DUT: df44cb3b
[TIME:    90] [DIV_3] - FAIL: Expect: 095ec150, DUT: 095ec14c
[TIME:   100] [DIV_3] - PASS: Expect: e0bc68a5, DUT: e0bc68a5

...

[TIME:   990] [DIV_3] - FAIL: Expect: d7bef4fd, DUT: d7bef4f4
[TIME:  1000] [DIV_3] - FAIL: Expect: 224c4213, DUT: 224c4225

=====
=====TEST SUMMARY=====
Total test cases:      100
Passed                :      6
Failed                :     94
Pass rate             : 6.00%
=====

Simulation complete via $finish(1) at time 1 US + 0
../01_tb/div_3_tb.sv:39      $finish;
xcelium> exit

```

Listing 3: Kết quả kiểm định cho bộ xấp xỉ chia 3.

Lí do mà scoreboard trả về số trường hợp PASS ít như vậy là do sai số từ những trường hợp ép kiểu và làm tròn của số real.

Tiến hành khảo sát, thống kê sự chênh lệch tối đa và tối thiểu của kết quả thiết kế so với giá trị Golden Model được sử dụng. Khảo sát trên 10000 trường hợp.

```
1 module div_3_tb_estimate();
2
3     parameter TEST_LOOP = 10000;
4     parameter real ESTIMATE = (2.0**-2) + (2.0**-4) + (2.0**-6) + (2.0**-8);
5
6     int value_real;
7
8     logic [31:0] logic_value, logic_result;
9
10    logic [31:0] operator_div, deviation;
11
12    int a = 0 ;
13    int b = 0 ;
14
15    int value_begin;
16    shortreal trans_value;
17
18    div_3_estimate dut_2 (
19        .i_a      (logic_value),
20        .o_result(logic_result)
21    );
22
23    initial begin
24        $shm_open("waves.shm");
25        $shm_probe("ASM") ;
26    end
27
28    initial begin
29        value_begin = 32'h7FFF_FFFF;
30        logic_value = value_begin;
31        trans_value = $itor(value_begin);
32        value_real = int'(trans_value * ESTIMATE);
33        #10
34        $display("TIME: %7t INPUT: %11d, OUTPUT: %11d, CAL_REAL_TO_INT: %11d, Deviation %11d", $time,
35            $signed(logic_value), $signed(logic_result), $signed(value_real), $signed(logic_result - value_real));
36
37        value_begin = 32'hFFFF_FFFF;
38        logic_value = value_begin;
39        trans_value = $itor(value_begin);
40        value_real = int'(trans_value * ESTIMATE);
41        #10
42        $display("TIME: %7t INPUT: %11d, OUTPUT: %11d, CAL_REAL_TO_INT: %11d, Deviation %11d", $time,
43            $signed(logic_value), $signed(logic_result), $signed(value_real), $signed(logic_result - value_real));
44
45        for(int i = 0; i < TEST_LOOP; i++) begin
46            value_begin = $urandom_range(0, 2**32 - 1);
47            logic_value = value_begin;
48            trans_value = $itor(value_begin);
49            value_real = int'(trans_value * ESTIMATE);
50            #10
51            find_max(logic_result - value_real);
52            find_min(logic_result - value_real);
53            $display("TIME: %7t INPUT: %11d, OUTPUT: %11d, CAL_REAL_TO_INT: %11d, Deviation %11d", $time,
54                $signed(logic_value), $signed(logic_result), $signed(value_real), $signed(logic_result - value_real));
55        end
56    end
```

```

54
55     $display("MAX Deviation = %11d, MIN Deviation = %11d", a, b);
56
57     $finish;
58 end
59
60 task automatic find_max(
61     input int value
62 );
63
64     a = (a <= value) ? value : a;
65
66 endtask
67
68 task automatic find_min(
69     input int value
70 );
71
72     b = (b > value) ? value : b;
73
74 endtask
75
76 endmodule

```

Listing 4: HDL mô tả thiết kế bộ xấp xỉ chia 3.

Nhóm thu được kết quả:

```

xcelium> run
TIME:      10 INPUT:  2147483647, OUTPUT:   713031679, CAL_REAL_TO_INT:
      713031680, Deviation           -1
TIME:      20 INPUT:      -1, OUTPUT:      -1, CAL_REAL_TO_INT:
      0, Deviation           -1
TIME:      30 INPUT: -1091929648, OUTPUT: -362554766, CAL_REAL_TO_INT:
      -362554750, Deviation        -16

...

TIME:  100010 INPUT:   834127514, OUTPUT:   276956401, CAL_REAL_TO_INT:
      276956393, Deviation           8
TIME:  100020 INPUT: -554033200, OUTPUT: -183956336, CAL_REAL_TO_INT:
      -183956341, Deviation          5
MAX Deviation =           21, MIN Deviation =          -22
Simulation complete via $finish(1) at time 100020 NS + 0
../01_tb/div_3_tb_estimate.sv:57      $finish;
xcelium> exit

```

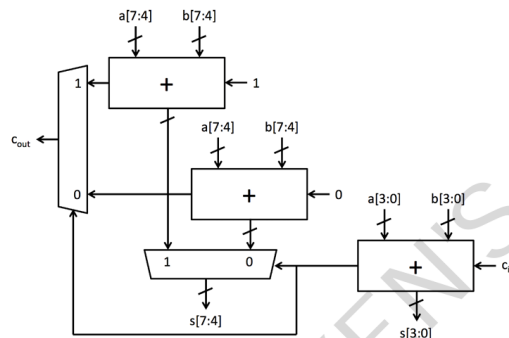
Listing 5: Kết quả kiểm định cho bộ xấp xỉ chia 3.

Sai số dao động trong khoảng $[-22;21]$ khi so sánh kết quả đầu ra của thiết kế với giá trị Golden Model. Đây là khoảng sai số có thể chấp nhận (sai lệch 5 bit đầu) vì thiết kế chỉ xấp xỉ giá trị của phép chia 3.

Câu 4

Thiết kế mạch tính tổng 2 số sử dụng giải thuật CSA (Carry Select Adder)

Hình vẽ sau mô tả mạch tổ hợp của giải thuật CSA áp dụng cho 2 số 8 bit. Trong đó, bộ cộng 4 bit có thể chọn CLA hoặc CRA.



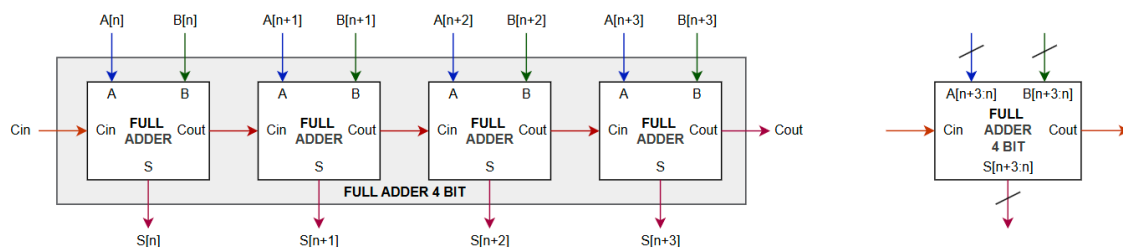
Hình 2: Bộ cộng CSA 8 bit.

Cho các standard cell là: cổng not, các cổng logic 2, 3, 4 ngõ vào, mux 2-1.

a) Thiết kế mạch theo phương pháp đã cho và chỉ được dùng các standard cell trên.

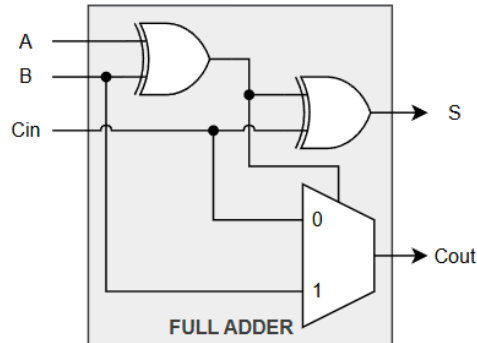
Bộ cộng theo kiến trúc CSA gồm một bộ cộng Full Adder 4 bit ban đầu ở đầu ghép với chuỗi các khối tính toán song song với hai giá trị ngõ vào Cin là 0 và 1, dựa vào Cout trước đó để lựa chọn kết quả và phép nhớ, từ đó giảm thời gian lan truyền phép nhớ so với kiến trúc CRA tiêu chuẩn.

Đầu tiên nhóm em sẽ thiết kế bộ cộng CRA trước, thiết kế của bộ cộng CLA đã được nhóm em trình bày ở Câu 3. Khối CRA được thiết kế bằng cách ghép 4 khối Full Adder nối tiếp nhau.



Hình 3: Bộ cộng CRA 4 bit.

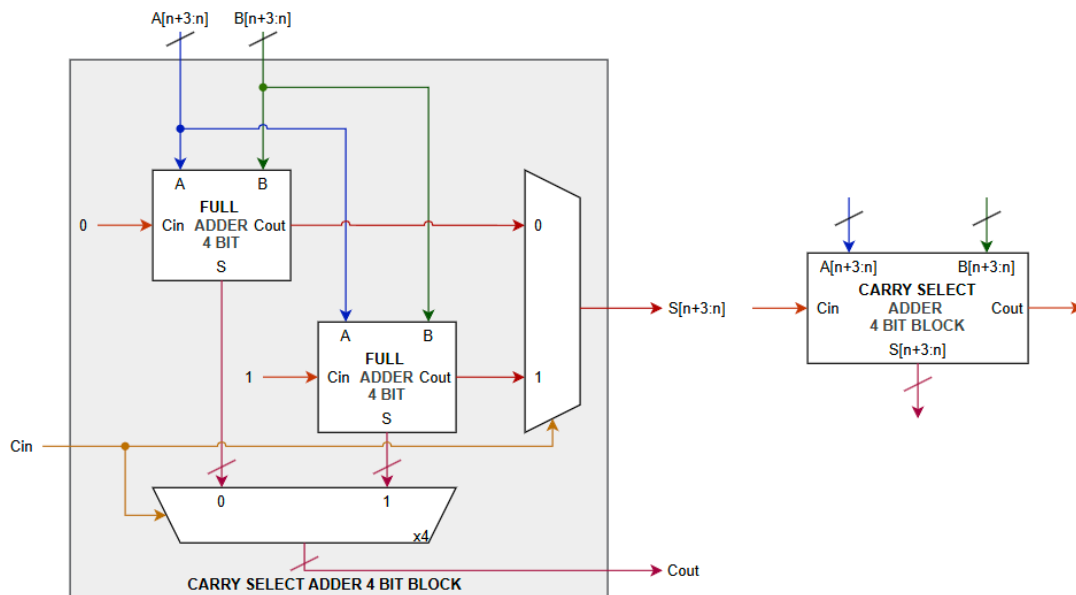
Với mỗi khối Full Adder, nhóm sử dụng thiết kế như hình:



Hình 4: Full Adder.

Thiết kế gồm 2 cell XOR 2 ngõ vào và 1 cell MUX 2-1. Kiểu thiết kế Full Adder này sẽ sử dụng ít cell hơn (2 cell) so với kiểu thiết kế Full Adder truyền thống, đồng thời số lượng cascade giảm còn 2 giúp thiết kế hoạt động ở tốc độ cao hơn.

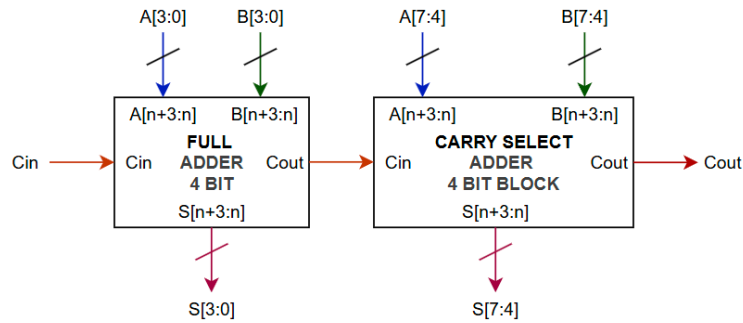
Tiếp theo, nhóm sẽ thiết kế khối tính toán song song.



Hình 5: Khối tính toán song song 4 bit.

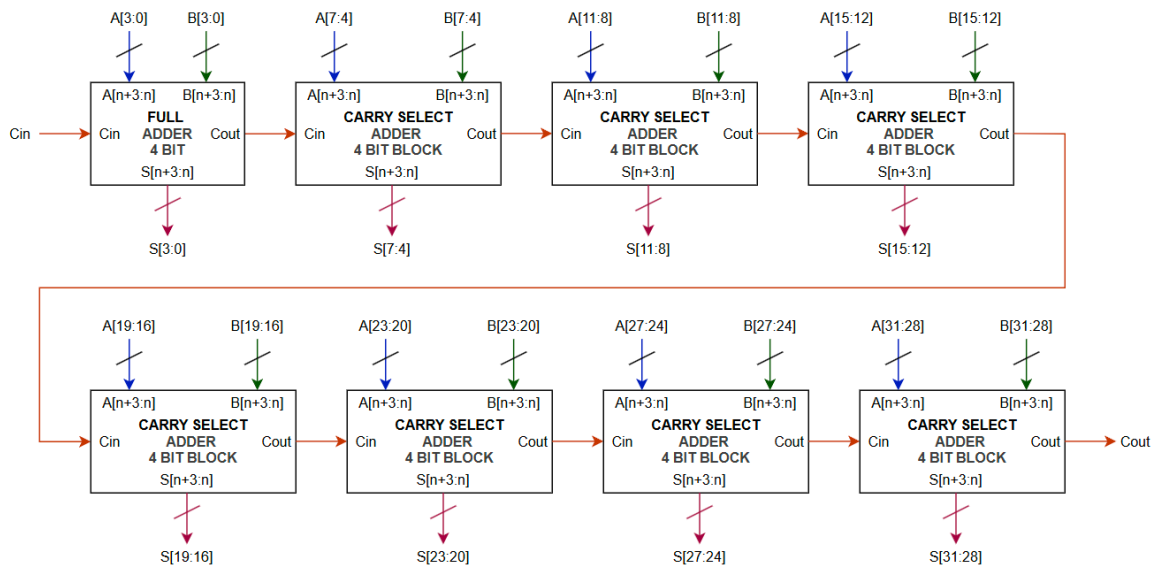
Khối tính toán song song gồm hai bộ Full Adder 4 bit (có thể là CLA hoặc CRA) và các bộ MUX 2-1 (1 để chọn cờ nhớ và 4 để lựa chọn kết quả tính toán).

Với yêu cầu từ đề bài, để thiết kế bộ cộng 8 bit theo giải thuật CSA, ta cần 1 bộ Full Adder 4 bit và 1 khối tính toán song song 4 bit.



Hình 6: Bộ cộng CSA 8 bit thiết kế.

Với yêu cầu mở rộng thiết kế bộ cộng 32 bit theo giải thuật CSA, ta cần 1 bộ Full Adder 4 bit và 7 khối tính toán song song 4 bit.



Hình 7: Bộ cộng CSA 32 bit.

b) Viết chương trình HDL mô tả mạch đã cho.

Dựa vào yêu cầu của đề bài, nhóm cần thiết kế bộ cộng 8 bit và 32 bit bằng giải thuật, điều đó đòi hỏi HDL mô tả phải có khả năng linh hoạt (có thể điều chỉnh tham số để hình thành các thiết kế khác nhau).

```
1 module fa_1b (  
2     input logic i_a ,  
3     input logic i_b ,  
4     input logic i_cin ,  
5     output logic o_s ,  
6     output logic o_cout  
7 );  
8  
9     logic a_xor_b;  
10    assign a_xor_b = i_a ^ i_b ;  
11    assign o_s = a_xor_b ^ i_cin ;  
12    assign o_cout = a_xor_b ? i_cin : i_b;  
13  
14 endmodule
```

Listing 6: HDL mô tả thiết kế Full Adder 1 bit.

```
1 module cra_4b (  
2     input logic [3:0] i_a ,  
3     input logic [3:0] i_b ,  
4     input logic i_cin ,  
5     output logic [3:0] o_s ,  
6     output logic o_cout  
7 );  
8  
9     logic [4:0] c;  
10    assign c[0] = i_cin;  
11    assign o_cout = c[4] ;  
12  
13    genvar i;  
14    generate  
15        for(i = 0; i < 4; i++) begin : gen_4_block  
16            fa_1b FA (  
17                .i_a (i_a[i]) ,  
18                .i_b (i_b[i]) ,  
19                .i_cin ( c[i]) ,  
20                .o_s (o_s[i]) ,  
21                .o_cout(c[i + 1])  
22            );  
23        end  
24    endgenerate  
25  
26 endmodule
```

Listing 7: HDL mô tả thiết kế CRA 4 bit.

(*) Thiết kế của khối CLA 4 bit đã được đề cập ở Câu 3.

```

1  module csa #(
2      parameter WIDTH    = 12, // WIDTH must %4 == 0 (and >= 4)
3      parameter CLA_CRA = 1    // 1 use CLA 4 bit , 0 use CRA 4 bit
4  )(
5      input  logic [WIDTH - 1:0] i_a    ,
6      input  logic [WIDTH - 1:0] i_b    ,
7      input  logic          i_cin    ,
8      output logic [WIDTH - 1:0] o_s    ,
9      output logic          o_cout
10 );
11
12     localparam LENGTH = WIDTH/4;
13
14     logic [WIDTH - 1:4] s_cin_1, s_cin_0;
15     logic [LENGTH - 1:0] cout_sel;
16     logic [LENGTH - 2:0] cout_cin_1, cout_cin_0;
17
18     assign o_cout = cout_sel[LENGTH - 1];
19
20     generate
21         if(CLA_CRA == 0)
22             cra_4b first_block (
23                 .i_a    (i_a[3:0]),
24                 .i_b    (i_b[3:0]),
25                 .i_cin  (i_cin) ,
26                 .o_s    (o_s[3:0]),
27                 .o_cout (cout_sel[0])
28             );
29
30         else
31             cla_4 first_block (
32                 .a    (i_a[3:0]),
33                 .b    (i_b[3:0]),
34                 .cin  (i_cin),
35                 .s    (o_s[3:0]),
36                 .cout (cout_sel[0])
37             );
38     endgenerate
39
40     genvar i;
41     generate
42         for(i = 1; i < LENGTH; i++) begin : gen_block
43             if(CLA_CRA == 0) begin
44                 cra_4b cin_1 (
45                     .i_a    ( i_a[(i*4)+:4]),
46                     .i_b    ( i_b[(i*4)+:4]),
47                     .i_cin  ( 1'b1 ),
48                     .o_s    (s_cin_1[(i*4)+:4]),
49                     .o_cout (cout_cin_1[i - 1])
50                 );
51
52                 cra_4b cin_0 (
53                     .i_a    ( i_a[(i*4)+:4]),
54                     .i_b    ( i_b[(i*4)+:4]),
55                     .i_cin  ( 1'b0 ),
56                     .o_s    (s_cin_0[(i*4)+:4]),
57                     .o_cout (cout_cin_0[i - 1])
58                 );
59             end
60
61             else begin
62                 cla_4 cin_1 (
63                     .a    ( i_a[(i*4)+:4]),

```

```

64         .b      (      i_b[(i*4)+:4]),
65         .cin    (      1'b1 ),
66         .s      (s_cin_1[(i*4)+:4]),
67         .cout(cout_cin_1[i - 1])
68     );
69
70     cla_4 cin_0 (
71         .a      (      i_a[(i*4)+:4]),
72         .b      (      i_b[(i*4)+:4]),
73         .cin    (      1'b0 ),
74         .s      (s_cin_0[(i*4)+:4]),
75         .cout(cout_cin_0[i - 1])
76     );
77 end
78
79 assign cout_sel[i]  = cout_sel[i - 1] ? cout_cin_1[i - 1] : cout_cin_0[i - 1];
80 assign o_s[(i*4)+:4] = cout_sel[i - 1] ? s_cin_1[(i*4)+:4] : s_cin_0[(i*4)+:4];
81
82 end
83 endgenerate
84 endmodule

```

Listing 8: HDL mô tả thiết kế CSA linh động.

Thiết kế gồm 2 tham số đầu vào là WIDTH và CLA-CRA.

- WIDTH là tham số quyết định thiết kế sinh ra phép cộng 4, 8,...,32,... bit, do đó WIDTH luôn phải chia hết cho 4.
- CLA-CRA là tham số quyết định bộ Full Adder 4 bit được sử dụng trong thiết kế là CLA 4 bit (CLA-CRA > 0) hoặc CRA 4 bit (CLA-CRA = 0).

c) Viết testbench cho mạch, thực hiện testbench với 100 mẫu và tính scoreboard của 100 mẫu đó.

Phương pháp kiểm định được sử dụng tương tự với Câu 3. Kiểm thử với 100 mẫu ngẫu nhiên a, b, Cin được đưa vào thiết kế.

```
1  module csa_tb;
2
3      parameter WIDTH      = 32 ; // WIDTH must %4 == 0 (and >= 4)
4      parameter CLA_CRA    = 0  ; // 1 use CLA, 0 use CRA
5      parameter TEST_LOOP  = 100;
6
7      logic [WIDTH - 1:0] a, b;
8      logic               cin;
9      logic [WIDTH - 1:0] s, s_expect;
10     logic                cout, cout_expect;
11
12     csa #(
13         .WIDTH(WIDTH),
14         .CLA_CRA(CLA_CRA)
15     ) DUT (
16         .i_a(a),
17         .i_b(b),
18         .i_cin(cin),
19         .o_s(s),
20         .o_cout(cout)
21     );
22
23     initial begin
24         $shm_open("waves.shm");
25         $shm_probe("ASM");
26     end
27
28     int test_count = 0;
29     int test_pass  = 0;
30
31     task printf_compare (input string type_tb, input int tb_value, input int dut_value);
32         $display("[TIME: %5t] [%s] - %4s: Expect: %8h, DUT: %8h ", $time, type_tb, (tb_value == dut_value) ?
33             "PASS" : "FAIL", tb_value, dut_value);
34         test_count = test_count + 1;
35         test_pass  = (tb_value == dut_value) ? test_pass + 1 : test_pass;
36     endtask
37
38     task report();
39         $display("\n=====");
40         $display("=====TEST SUMMARY=====");
41         $display("Total test cases: %6d ", test_count);
42         $display("Passed           : %6d ", test_pass);
43         $display("Failed           : %6d ", test_count - test_pass);
44         $display("Pass rate       : %0.2f%%", (test_pass * 100.0) / (test_count));
45         $display("=====");
46     endtask
47
48     initial begin
49         for (int i = 0; i < TEST_LOOP; i++) begin
50             a = $urandom_range(0, 2**WIDTH - 1);
51             b = $urandom_range(0, 2**WIDTH - 1);
52             cin = $urandom_range(0, 1);
53             {cout_expect, s_expect} = a + b + cin;
54             #10
55             $display("[TIME: %5t] a = %8h, b = %8h, Cin = %1h", $time, a, b, cin);
```

```

55     printf_compare("CSA - RESULT", s_expect, s);
56     printf_compare(" CSA - COUT ", cout_expect, cout);
57     $display("\n");
58     end
59
60     report();
61     #10
62     $finish;
63     end
64 endmodule

```

Listing 9: Kiểm định giành cho CSA.

Điểm khác biệt so với Câu 3 là testbench xuất hiện thêm tham số điều chỉnh WIDTH nhằm để thuận tiện trong việc kiểm định hai loại thiết kế là CSA 8 bit và 32 bit.

Vì testbench tính PASS/FAIL của kết quả và cờ nhớ riêng nên số trường hợp PASS đạt 200 so với yêu cầu là 100.

Kết quả:

```

xcelium> run
[TIME: 10] a = 0000001b, b = 0000003b, Cin = 0
[TIME: 10] [CSA - RESULT] - PASS: Expect: 00000056, DUT: 00000056
[TIME: 10] [ CSA - COUT ] - PASS: Expect: 00000000, DUT: 00000000

[TIME: 20] a = 0000000e, b = 0000007a, Cin = 0
[TIME: 20] [CSA - RESULT] - PASS: Expect: 00000088, DUT: 00000088
[TIME: 20] [ CSA - COUT ] - PASS: Expect: 00000000, DUT: 00000000

...

[TIME: 1000] a = 0000007a, b = 00000058, Cin = 1
[TIME: 1000] [CSA - RESULT] - PASS: Expect: 000000d3, DUT: 000000d3
[TIME: 1000] [ CSA - COUT ] - PASS: Expect: 00000000, DUT: 00000000

=====
=====TEST SUMMARY=====
Total test cases:      200
Passed                :    200
Failed                :         0
Pass rate             : 100.00%
=====

Simulation complete via $finish(1) at time 1010 NS + 0
../01_tb/csa_tb.sv:62      $finish;
xcelium> exit

```

Listing 10: Kết quả kiểm định cho thiết kế CSA 8 bit.

```

xcelium> run
[TIME:    10] a = 1b4c0964, b = 3b3cd304, Cin = 0
[TIME:    10] [CSA - RESULT] - PASS: Expect: 5688dc68, DUT: 5688dc68
[TIME:    10] [ CSA - COUT ] - PASS: Expect: 00000000, DUT: 00000000

[TIME:    20] a = 0e190828, b = 7a369fc6, Cin = 0
[TIME:    20] [CSA - RESULT] - PASS: Expect: 884fa7ee, DUT: 884fa7ee
[TIME:    20] [ CSA - COUT ] - PASS: Expect: 00000000, DUT: 00000000

...

[TIME:   1000] a = 7a2f41da, b = 586b58d4, Cin = 1
[TIME:   1000] [CSA - RESULT] - PASS: Expect: d29a9aaf, DUT: d29a9aaf
[TIME:   1000] [ CSA - COUT ] - PASS: Expect: 00000000, DUT: 00000000

=====
=====TEST SUMMARY=====
Total test cases:      200
Passed                :    200
Failed                :         0
Pass rate             : 100.00%
=====

Simulation complete via $finish(1) at time 1010 NS + 0
../01_tb/csa_tb.sv:61          $finish;
xcelium> exit

```

Listing 11: Kết quả kiểm định cho thiết kế CSA 32 bit.